

Name : Bhavya rana
Sap id : 590034689
Batch : 15

1. Algorithm

Main Program

1. Start.
2. Declare an integer variable `choice`.
3. Display the menu:
 - 1 – Check Strong Number
 - 2 – Check Armstrong Number
 - 3 – Check Harshad Number
 - 4 – Check Duck Number
4. Read the user's choice.
5. Use `switch(choice)`:
 - Case 1: Call `strongNumber()`.
 - Case 2: Call `armstrongNumber()`.
 - Case 3: Call `harshadNumber()`.
 - Case 4: Call `duckNumber()`.
 - Default: Display "Invalid choice".
6. Stop.

Function 1: Strong Number

A number is **Strong** if the sum of the factorials of its digits equals the original number. Example:
 $145 = 1! + 4! + 5!$.

1. Read a positive integer `n`.
2. Store `n` in `temp`.
3. Initialize `sum = 0`.
4. Extract each digit using `% 10`.
5. Calculate the factorial of the digit.
6. Add the factorial to `sum`.
7. Remove the last digit using `/ 10`.
8. Repeat until all digits are processed.
9. If `sum == n`, display that it is a Strong number; otherwise, display that it is not.

Function 2: Armstrong Number

A number is **Armstrong** if the sum of each digit raised to the power of the number of digits equals the original number. Example: $153 = 1^3 + 5^3 + 3^3$.

1. Read a positive integer `n`.
2. Count the number of digits.
3. Extract each digit.
4. Calculate `digit` raised to the number of digits.
5. Add it to `sum`.
6. Repeat for all digits.
7. If `sum == n`, display that it is an Armstrong number; otherwise, display that it is not.

Function 3: Harshad Number

A number is **Harshad** if it is divisible by the sum of its digits. Example: 18, because $1 + 8 = 9$ and $18 \% 9 = 0$.

1. Read a positive integer `n`.
2. Find the sum of its digits.
3. Calculate `n % sum`.
4. If the remainder is 0, display that it is a Harshad number.
5. Otherwise, display that it is not.

Function 4: Duck Number

A positive integer is a **Duck number** if it contains at least one 0 among its digits. Example: 1023.

1. Read a positive integer `n`.
2. Extract its digits one by one.
3. Check whether any digit is 0.
4. If a zero is found, mark the number as a Duck number.
5. Otherwise, it is not a Duck number.

2. Test Case Table

Test Case	Menu Choice	Input	Expected Output	Actual Output	Result
1	1	145	145 is a Strong number	145 is a Strong number	Pass
2	1	123	123 is not a Strong number	123 is not a Strong number	Pass
3	2	153	153 is an Armstrong number	153 is an Armstrong number	Pass
4	2	123	123 is not an Armstrong number	123 is not an Armstrong number	Pass
5	3	18	18 is a Harshad number	18 is a Harshad number	Pass
6	3	19	19 is not a Harshad number	19 is not a Harshad number	Pass
7	4	1023	1023 is a Duck number	1023 is a Duck number	Pass
8	4	1234	1234 is not a Duck number	1234 is not a Duck number	Pass

3. C Program

```
#include <stdio.h>

/* Function declarations */
void strongNumber(void);
void armstrongNumber(void);
void harshadNumber(void);
void duckNumber(void);

int main(void)
{
    int choice;

    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strongNumber();
            break;

        case 2:
            armstrongNumber();
            break;

        case 3:
            harshadNumber();
            break;

        case 4:
            duckNumber();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

```
/* Function to check Strong Number */
void strongNumber(void)
{
    int n, temp, digit;
    int sum = 0, fact, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;

        fact = 1;

        for (i = 1; i <= digit; i++)
        {
            fact = fact * i;
        }

        sum = sum + fact;
        temp = temp / 10;
    }

    if (sum == n)
        printf("%d is a Strong number.\n", n);
    else
        printf("%d is not a Strong number.\n", n);
}

/* Function to check Armstrong Number */
void armstrongNumber(void)
{
    int n, temp, digit;
    int digits = 0, sum = 0;
    int power, i;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    /* Count number of digits */
```

```
while (temp > 0)
{
    digits++;
    temp = temp / 10;
}

temp = n;

while (temp > 0)
{
    digit = temp % 10;

    power = 1;

    for (i = 1; i <= digits; i++)
    {
        power = power * digit;
    }

    sum = sum + power;
    temp = temp / 10;
}

if (sum == n)
    printf("%d is an Armstrong number.\n", n);
else
    printf("%d is not an Armstrong number.\n", n);
}

/* Function to check Harshad Number */
void harshadNumber(void)
{
    int n, temp, digit;
    int sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;
        sum = sum + digit;
        temp = temp / 10;
    }
}
```

```
    }

    if (sum != 0 && n % sum == 0)
        printf("%d is a Harshad number.\n", n);
    else
        printf("%d is not a Harshad number.\n", n);
}

/* Function to check Duck Number */
void duckNumber(void)
{
    int n, temp, digit;
    int hasZero = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &n);

    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;

        if (digit == 0)
        {
            hasZero = 1;
            break;
        }

        temp = temp / 10;
    }

    if (hasZero == 1)
        printf("%d is a Duck number.\n", n);
    else
        printf("%d is not a Duck number.\n", n);
}
```

4. Compilation

The program was compiled using GCC with warnings enabled:

```
gcc -std=c11 -Wall -Wextra -pedantic main.c -o mai
```

Compilation result: Successful — no compilation errors.

5. Sample Executions

Strong Number

```
Last login: Fri Sep 11 19:34:47 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day18.cp
bhavya@Bhavyas-MacBook-Air ~ % clang day18.cp -o day18
bhavya@Bhavyas-MacBook-Air ~ % ./day18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.
bhavya@Bhavyas-MacBook-Air ~ %
```

Armstrong number

```
Last login: Fri Sep 11 19:34:47 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day18.cp
bhavya@Bhavyas-MacBook-Air ~ % clang day18.cp -o day18
bhavya@Bhavyas-MacBook-Air ~ % ./day18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.
bhavya@Bhavyas-MacBook-Air ~ % ./day18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.
bhavya@Bhavyas-MacBook-Air ~ %
```

Harshad number

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.
```

Duck number

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day18
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 4
Enter a positive integer: 1023
1023 is a Duck number.
bhavya@Bhavyas-MacBook-Air ~ %
```

6. Testing Result

I compiled the program and executed **all 8 prepared test cases**. All 8 produced the expected output.

Total test cases: 8

Passed: 8

Failed: 0

Overall result: PASS 